

StartupBench: Benchmarking General-Purpose Agents on Market-Validated Workflows

Author1^{1,2,*,†}, Author2¹, Author3^{1,2,*}, Author4¹, Author5^{1,†}

¹ByteDance Seed, ²Peking University

*Work done at ByteDance Seed, [†]Corresponding authors

Abstract

Recent advances in foundation models and agent frameworks have significantly improved the ability of AI systems to perform complex tasks. However, existing benchmarks largely evaluate capabilities selected by researchers, leaving open a fundamental question: to what extent can current agents accomplish the end-to-end work that people actually wish to delegate to AI? We argue that successful startup-agent products provide a unique lens into this question, as they represent market-validated services built around real user demand. To study this gap, we introduce **StartupBench**, a benchmark of end-to-end workflows derived from extensive analysis of startup-agent products, product demonstrations, and user interviews across diverse professional domains. The resulting tasks capture real-world work that users actively adopt and pay for AI systems to perform, ranging from research and analysis to content creation and business operations. We evaluate frontier general-purpose agents on StartupBench and find that many market-validated workflows remain challenging despite recent advances in foundation models and agent frameworks. Through detailed task and failure-mode analyses, we identify recurring limitations in long-horizon execution, implicit requirement understanding, workflow orchestration, and deliverable quality. Our results reveal a substantial gap between the work users seek to delegate and the capabilities of current general-purpose agents, highlighting important directions for future agent training, evaluation, and development.

Date: June 30, 2026

Correspondence: Author1 at xxx@bytedance.com, Author5 at [xxx](mailto:xxx@bytedance.com)

Project Page: [xxx](https://github.com/ByteDance/StartupBench)

1 Introduction

Large language models (LLMs) and agents are increasingly being deployed as systems for task execution rather than only as interfaces for information retrieval or question answering. Existing evaluations have provided important measurements of knowledge, reasoning, coding, and tool-use abilities [2, 8, 16], often in relatively isolated problem settings. While these abilities remain fundamental, they only partially characterize the value of deployed agent systems. In practical use, users expect AI systems to complete concrete tasks, reduce manual effort, and produce usable deliverables under realistic workflow constraints.

This shift motivates a central evaluation question: to what extent can current models and agents complete the kinds of work that users already seek to delegate to AI systems? We argue that this ability is critical for assessing practical usefulness, because real-world AI adoption provides an empirical signal of what tasks

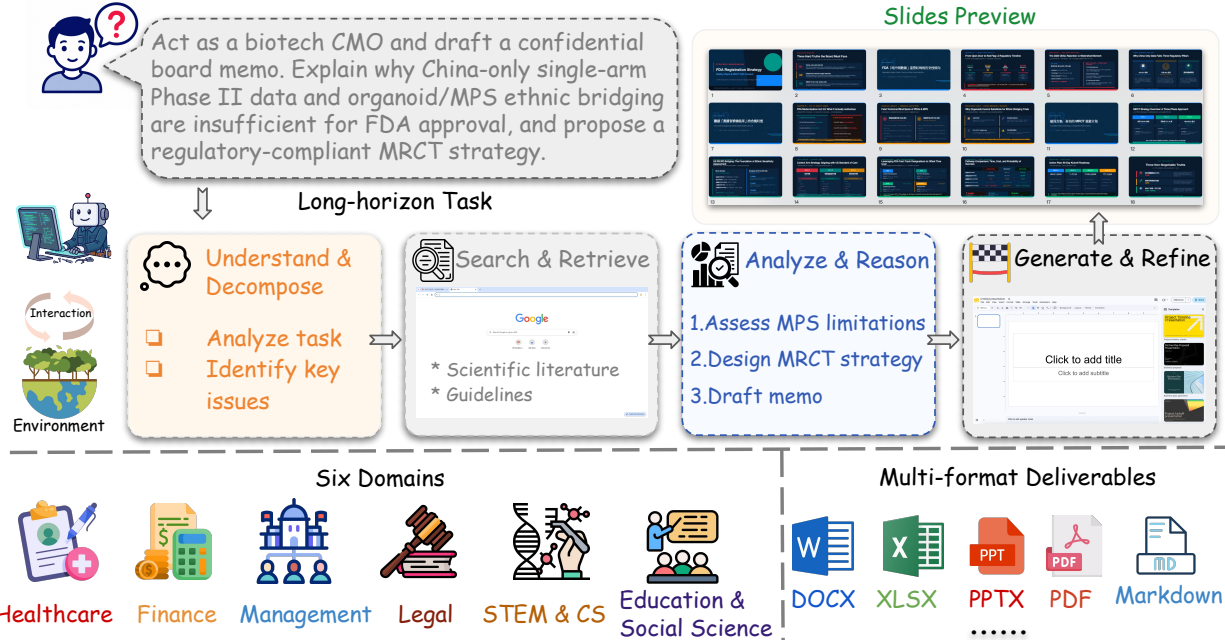


Figure 1 Overview of StartupBench: real AI-product workflows, long-horizon agent execution, six-domain coverage, and multi-format deliverable generation.

users find valuable enough to automate. In practice, users adopt AI systems not merely to demonstrate isolated capabilities, but to complete concrete work and produce usable deliverables under realistic constraints. Moreover, workflows that achieve broad adoption, sustained usage, or commercial success offer additional evidence that they address genuine user needs. The tasks that users choose to delegate to AI therefore reveal practical demand: they reflect what kinds of work users care about, what outputs they expect, and what requirements those outputs must satisfy. Evaluating AI systems on such tasks consequently measures not only individual capabilities such as reasoning or tool use, but also the ability to transform these capabilities into useful work products.

However, existing agent benchmarks provide limited coverage of this demand-driven evaluation setting. Many benchmarks focus on general reasoning, web navigation, tool use, software engineering, or simulated environments. While these settings have significantly advanced the evaluation of model capabilities, they are not always grounded in workflows that users already perform with AI systems in practice. Other benchmarks evaluate isolated subtasks rather than the end-to-end deliverables that users ultimately require. As a result, strong performance on existing benchmarks does not necessarily imply successful completion of real-world work. This gap motivates the **need for evaluations derived from practical AI adoption, where tasks are defined not solely by researchers, but also by the workflows that users already choose to delegate to specified AI systems.**

To address this gap, we introduce **StartupBench**, a survey- and interview-driven benchmark for evaluating whether models and general-purpose agents can complete real-world workflows derived from AI-native startups and their users. The central premise is that AI-native startups can serve as an empirical source for discovering practically relevant agent tasks, since these companies operationalize model capabilities into concrete products and adapt them to enterprise-user demand. We first survey AI agent startups to identify productized workflows, and then analyze product demonstrations and conduct user interviews to understand how customers use these products, what needs they address, and what deliverables users expect. Based on these findings, we abstract representative workflows into benchmark tasks with specified inputs, expected outputs, and evaluation criteria, and apply multi-stage quality control to ensure that the resulting tasks are realistic, challenging, and assessable.

We evaluate a diverse set of representative foundation models and general-purpose agents on StartupBench. Our results show that, while current systems can successfully complete parts of many startup-derived workflows, they remain far from delivering complete, production-ready outputs under a general-purpose agent harness. Compared with specialized startup-agent systems, general-purpose agents still exhibit a substantial capability gap on realistic business tasks. Our analysis further attributes these failures primarily to deficiencies in complex instruction following, domain-specific knowledge and professional conventions, and reliable execution under realistic workflow constraints. Together, these findings highlight key bottlenecks in current general-purpose agents and suggest concrete directions for improving future foundation models, agent systems, and evaluation methodologies.

Our contributions are as follows:

- We propose a survey- and interview-driven methodology for constructing agent benchmarks from real AI adoption scenarios.
- We construct StartupBench, a benchmark built through AI-native startup research, enterprise-user interviews, user-oriented task abstraction, expert data construction, and multi-stage quality control.
- We present a comprehensive empirical study of current foundation models and general-purpose agents on startup-derived workflows, providing a practical evaluation signal for assessing whether AI agents are progressing from answering questions toward reliably completing real work.

2 Related Work

2.1 Commercial AI agents and Vertical Workflows

Recent advances in foundation models have enabled AI agents to gradually acquire planning, tool-use, file-processing, and multi-step execution capabilities [12, 20], pushing AI systems from general conversational assistants toward workflow-oriented products. In practice, commercial AI products increasingly package these capabilities into vertical workflows such as document processing, data analysis, financial research, enterprise operations, and knowledge work. This trend suggests that real-world AI value is not determined only by isolated model capabilities, but also by whether these capabilities can be productized into useful work products. However, existing academic benchmarks rarely use commercial adoption itself as a source of task discovery. StartupBench addresses this gap by deriving tasks from market-validated AI-native startups and their deep users, moving task construction from researcher-defined settings toward demand-driven workflow construction.

2.2 End-to-end evaluation of AI agents

Agent benchmarks have evolved from capability evaluation toward end-to-end task completion. Early benchmarks mainly focus on tool use, multi-step reasoning, and general assistant abilities, as in AgentBench and GAIA [6, 8]. Later work extends evaluation to more realistic interactive environments, including web interaction and computer-use tasks [17, 23]. More recent benchmarks further move toward professional work and economically meaningful tasks, covering enterprise software, simulated workplaces, occupational tasks, and long-horizon workflows [1, 3, 7, 11, 18, 24]. These works reflect a clear transition from isolated capability tests to realistic task-completion evaluation. StartupBench follows this direction, but differs in its task source: rather than relying primarily on public environments, occupational taxonomies, or simulated workplaces, it derives tasks from AI-native startup products, ToB user needs, and expected business deliverables.

2.3 Agent-as-Judge for complex deliverable evaluation

As agent tasks shift from short answers to heterogeneous deliverables such as reports, spreadsheets, slides, code patches, and structured files, evaluation must move from capability-centric scoring to deliverable-centric assessment[21]. LLM-as-a-Judge has been widely used for open-ended response evaluation, with MT-Bench and Prometheus studying preference-based or rubric-conditioned judging [4, 22]. For agentic workflows, however, judging often requires evidence retrieval, file inspection, state verification, and constraint checking over final artifacts. Agent-as-a-Judge extends LLM judging by equipping evaluators with tools and environment interaction, while AJ-Bench further studies judge agents on information acquisition, state verification, and

Table 1 Comparison of StartupBench with existing benchmarks. E2E denotes end-to-end workflow completion; Item-wise Eval. denotes criterion-level judging. ✓, ✗, and △ indicate full, no, and partial support.

Benchmark	Task Source	Input Format	Final Output	E2E	Item-wise Eval.	Process Logic	Open-ended	Evaluation Method
<i>Other Benchmarks</i>								
GAIA [8]	Manual	NL(+Files)	Answer	△	✗	✗	✗	Exact match
OSWorld [17]	Real+Manual	Actions +NL	Actions	△	✗	✗	✗	Execution-based
DAComp [5]	Real+Manual	Data+NL	SQL repo / Report	✓	✗	✗	✓	Exec.+LLM-judge
<i>Office / Workplace Deliverable Benchmarks</i>								
GDPval [11]	Manual	Files+NL	Work products	✓	✗	✗	△	Expert pairwise
Workspace-Bench [15]	Sim.+Manual	Files+NL	Workspace files	✓	✗	✗	✓	Agent-as-Judge
OneMillionBench [19]	Expert-authored	NL(+Web)	Answer/Report	△	✗	△	✓	LLM-judge
Agents' Last Exam [14]	Occupation-guided	Soft.+NL	Verifiable artifacts	✓	✗	△	△	Outcome verifiers
OfficeQA Pro [10]	Corpus-grounded	Docs+NL	Answer	✗	✗	✗	✗	Answer match
StartupBench(Ours)	Market-vetted + Expert-built	Files(Noisy)+ NL+Web	Multi-format deliverables	✓	✓	✓	✓	Agent-as-Judge



Figure 2 StartupBench data construction pipeline. We select market-validated startup agents, collect real user workflows, construct task artifacts and instances, and calibrate quality and difficulty through multi-stage review.

process verification [13, 25]. StartupBench adopts this deliverable-centric evaluation paradigm by combining expert rubrics with automated judge agents that inspect original artifacts and evidence views, evaluating whether agents can transform model capabilities into usable work products.

3 StartupBench

StartupBench is designed to evaluate whether models and agents can complete realistic workflows that users already delegate to AI products. This objective requires data that reflects authentic user demand while supporting controlled evaluation. We therefore require each task to be realistic, answerable, evaluable, and discriminative: it should originate from actual AI product usage, provide sufficient information for a valid solution, specify clear output requirements and assessment criteria, and remain challenging enough to distinguish current models and agents.

Neither manual task design nor direct extraction from product demonstrations is sufficient to obtain such data. The former is controllable but may be detached from real demand, while the latter is realistic but often lacks context, success criteria, or reproducible specifications. We therefore adopt a survey- and interview-driven multi-stage pipeline. We first survey market-validated AI-native startups, interview deep users to identify usage contexts, task goals, and expected deliverables, recruit domain experts to convert validated scenarios

into benchmark instances, and finally apply quality control for realism, answerability, evaluability, and discriminative difficulty.

3.1 Dataset Construction

3.1.1 Startup Survey for Market-Validated Agent Scenarios

We first survey AI-native startups that build agent products across diverse domains. To identify workflows with market validation, we retain startups with more than USD 1M in funding and require evidence of real adoption, either through paid usage or substantial user traction. Funding provides a coarse signal of market expectation, while payment or user-scale evidence indicates that the product is used beyond exploratory demonstrations. The output of this stage is a pool of candidate products and workflows for user interviews. During this stage we collect 20+ start-up agents as the basis for further navigation.

3.1.2 User Interviews for Scenario and Requirement Discovery

For each candidate Startup-product, we interview deep users of the corresponding agent from different domains to recover practical usage beyond public product descriptions. The interviews elicit the usage context, user goal, input information, expected deliverable, success criteria, and common constraints. We summarize these findings into scenario specifications and representative demo cases, which serve as anchors for later annotation stage. During this stage we interview over 30 deep users and collected at least one demo case for each candidate Agent.

3.1.3 Expert Construction of Domain Tasks

Given the interview-derived scenarios and example tasks, we further recruit domain experts to instantiate benchmark instances that faithfully reflect real-world agent workflows while remaining suitable for controlled evaluation.

Each benchmark instance is formally defined as consisting of three components:

- **Input query:** The input query specifies the user instruction that initiates the task. Derived from authentic scenarios of the work of annotators, the input query is supposed to reflect realistic user intent, including task background, implicit constraints and expected deliverables.
- **Task workspace:** The task workspace provides a self-contained execution environment for the agent. Each workspace includes all necessary input artifacts (e.g., files, datasets, or structured resources) required to complete the task, and ensures that the agent has access to sufficient information to solve the problem without relying on external or hidden knowledge sources. This design enables reproducible evaluation under controlled conditions while preserving realistic workflow structure.
- **Evaluation rubrics:** The evaluation rubrics define a set of fine-grained criteria used to assess task completion quality. Due to the complexity and multi-faceted nature of real-world workflows, each task is evaluated using multiple rubric items that capture correctness, completeness, constraint satisfaction, and domain-specific quality dimensions.

Together, these components preserve the structure of real-world agent workflows while enabling systematic and objective evaluation. Tasks may require information synthesis, constraint following, structured generation, or domain-specific reasoning and judgment.

3.1.4 Quality Control

To ensure benchmark quality and consistency, every tasks undergoes the following multi-stage quality control process:

Expert Cross Validation. Each task is independently reviewed by at least one another domain expert with relevant professional experience. Rather than focusing only on annotation consistency, reviewers validate the realism and correctness of the task from multiple complementary perspectives.

- **Task authenticity.** Reviewers verify the authenticity of both the task description and the accompanying workspace. This includes checking that the input query reflects realistic user requests, that all provided materials are factually correct, and that no domain-specific knowledge errors exist.
- **Workflow fidelity.** Reviewers examine whether the task faithfully represents real-world workflows in the corresponding profession. Tasks that deviate from practical working procedures or fail to reflect genuine job responsibilities are revised or discarded.
- **Rubric validity.** Reviewers inspect the evaluation rubrics to ensure that the assessed capabilities are meaningful for the target workflow, that rubric items are clear and unambiguous, and that their relative weights reasonably reflect task priorities.
- **Ground-truth verification.** Reviewers validate the reference solution (ground truth) by evaluating it against the finalized rubrics. The ground-truth deliverable is required to achieve a full score. If inconsistencies are identified between the reference solution and the evaluation criteria, both the solution and the rubrics are revised until they are mutually consistent.

Difficulty Control. To ensure that the benchmark remains sufficiently discriminative for evaluating general-purpose agents, we perform an empirical difficulty calibration procedure based on the performance of existing frontier models. Rather than estimating task difficulty heuristically, we assess each candidate task through pilot executions under a unified general-agent harness.

Specifically, for every candidate task, we randomly sample several frontier models to complete the task using the same execution harness adopted in the final evaluation. Domain experts then inspect the generated outputs according to the predefined evaluation rubrics. Tasks that are consistently completed with high quality across different models are considered to provide limited discriminative value and are therefore excluded from the benchmark. The pilot evaluation also serves as an additional quality assurance stage. When experts determine that poor model performance is caused by deficiencies in the task itself—such as ambiguous user instructions, incomplete workspace information, or inadequately specified evaluation criteria—rather than the intrinsic difficulty of the task, the corresponding query, workspace, or rubrics are revised and re-validated before the task is admitted into the benchmark.

3.2 Statistics for StartupBench

StartupBench contains 97 real-world workflow tasks across 6 top-level domains: medical, finance, legal, business and management, STEM & computer science, and education & humanities. These tasks are designed to reflect deliverable-oriented workflows in realistic office settings, requiring agents to understand task contexts, follow complex constraints, process heterogeneous files, and produce usable work products. For evaluation, each task is evaluated using an average of 25.3 fine-grained rubrics, enabling detailed assessment of complex, long-horizon workflows and diverse work deliverables. The summary of the domain coverage and main output formats of StartupBench is shown at Table 2.

3.3 Automatic Evaluation of StartupBench Tasks

A **STARTUP** task can be defined as a triple $\mathcal{T} = (q, \mathcal{E}, \mathcal{R})$. The query q is a natural-language statement of the task. The environment $\mathcal{E}$ denotes the complete workspace provided to the agent, including the multimodal source files $\{a_1, \dots, a_m\}$ (e.g., PDF, DOCX, XLSX, PPTX, and images) together with any additional resources and interaction setting required to complete the task. The rubric $\mathcal{R} = \{(p_i, w_i)\}_{i=1}^n$ is an ordered checklist, where each item consists of a natural-language scoring point p_i and a positive importance weight w_i .

Given $(q, \mathcal{E})$, the model under test returns answer deliverables $\mathcal{D}$. To help the judge better understand the produced deliverables and their structure, we first build a lightweight evidence view $\mathcal{V}(\mathcal{D})$ that surfaces, per file, two fixed kinds of cues: the extracted content—plain text and embedded media unpacked from the file—and the page renders—rasterised screenshots of each page or slide that let the judge see the actual visual layout. These extractions are hints, not replacements: the judge still consults the original files when needed. This two-level structure lightens the judge’s burden and thereby cuts the long-context and OCR/parsing errors that would otherwise corrupt the verdicts.

Table 2 Statistics of StartupBench. The benchmark contains 97 real-world workflow tasks across six top-level domains and requires diverse deliverable formats.

Category	Count	Percentage (%)
<i>Overall statistics</i>		
Total tasks	97	100.0
Top-level domains	6	–
<i>Domain distribution</i>		
Medical	21	21.6
Finance	18	18.6
Legal	16	16.5
Business & Management	19	19.6
STEM & Computer Science	16	16.5
Education & Humanities	7	7.2

Output formats: DOCX, XLSX, PPTX, PDF, Markdown, images, and text-based deliverables.

Following **Agent-as-a-Judge**, each call is a small agent workflow rather than a single forward pass, but we push that idea down to per-item granularity: every scoring point gets its own tool-call budget, context window and failure domain. This matters because STARTUP rubrics are large and isolating them keeps one item’s failure from contaminating the rest, so the aggregate score carries lower bias. The judge then renders a binary verdict $y_i \in \{0, 1\}$ with a written justification e_i for each rubric item, and the task score is the importance-weighted pass rate.

$$(y_i, e_i) = \text{AgentJudge}(J; q, \mathcal{A}, \mathcal{D}, \mathcal{V}(\mathcal{D}), p_i), \quad (1)$$

$$\text{score}(\mathcal{D}, \mathcal{T}) = \frac{\sum_{i=1}^n w_i y_i}{\sum_{i=1}^n w_i} \in [0, 1]. \quad (2)$$

4 Experiments

4.1 Experimental Setup

For each task, the worker agent is initialized in the task-specific workspace together with the corresponding input query. The agent solves the task end-to-end, and the generated deliverables under the designated output directory are collected as the task output. Following the evaluation protocol described in the previous section, the output, together with the task specification, workspace context, and rubric set, is provided to the judge agent for automatic evaluation.

We benchmark 7 frontier models spanning both closed- and open-source families: (i) **closed-source**: GPT-5.5 [9], Gemini-3.1-pro, Seed-2.0-pro; (ii) **open-source**: Deepseek-v4-pro, Kimi-2.6-thinking, Qwen-3.6-max-thinking, GLM-5.1. All worker agents are executed under the same NANOBOT harness with an identical tool configuration. Each run is capped at a maximum of 200 interaction steps. For evaluation, the judge agent also runs under the same NANOBOT harness to ensure a consistent execution environment, while using GPT-5.5 as the underlying judge model. Apart from the continuous task score, we also report **pass@1**, which measures the fraction of tasks successfully completed under a single attempt. A task is considered successful if its final score is at least 90; otherwise, it is counted as a failure.

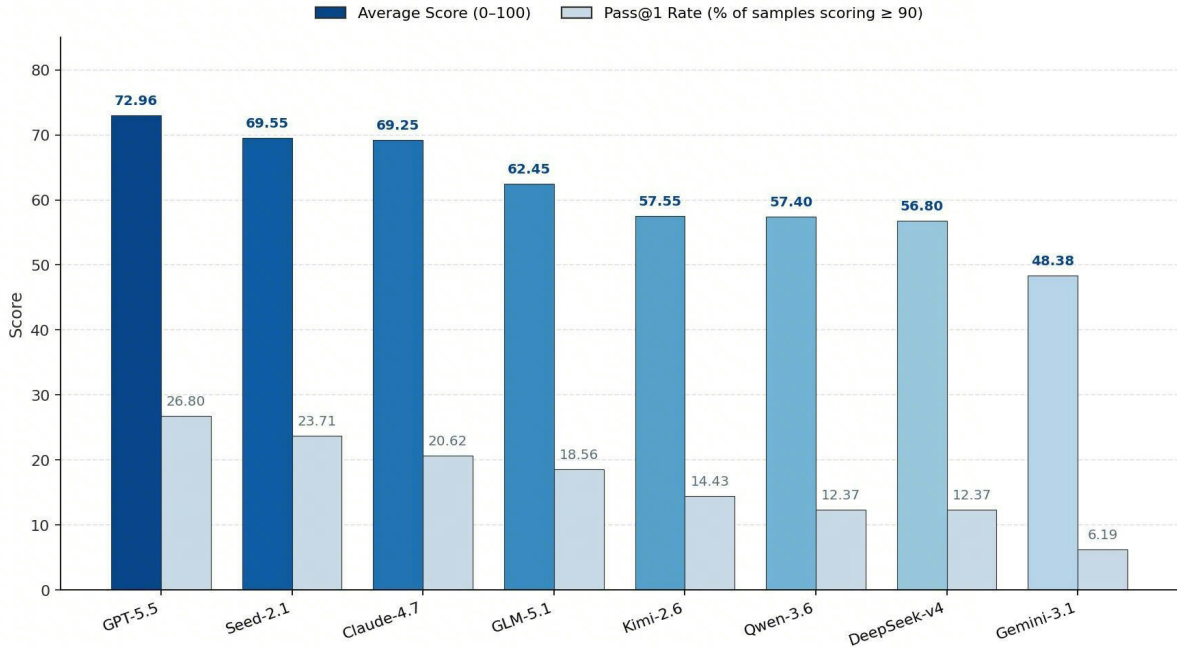


Figure 3 Leaderboard of StartupBench.

4.2 Main Results

Startup task remains far from saturated under a general-purpose agent harness. Although the strongest model (GPT-5.5) achieves an average score of 72.9, no model successfully completes even one third of the benchmark under the strict acceptance criterion (score ≥ 90). More broadly, the average scores of most models lie between roughly 50 and 70, indicating that current agents are generally capable of making substantial progress toward solving real-world professional workflows. However, this partial progress rarely translates into fully satisfactory deliverables: all models exhibit substantially lower pass@1 than average score, suggesting that end-to-end task completion remains a major bottleneck.

The gap between the two metrics reveals that the remaining challenge is no longer initiating or partially executing complex workflows, but consistently producing deliverables that satisfy all practical requirements. In other words, current general-purpose agents can often complete a large fraction of a user’s request, yet still fail to produce outputs that are directly usable without further human revision. This observation helps explain why specialized vertical agents continue to provide practical value in many market-validated workflows, while also highlighting a clear direction for future foundation models: improving the reliability and completeness of end-to-end task execution rather than merely increasing partial task capability. We further investigate the underlying causes of these failures in Section 4.4.2.

Performance varies substantially across professional domains, revealing that current general-purpose agents do not exhibit uniform competence across real-world workflows. Among the six domains, Medical is consistently the easiest: every model achieves an average score above 50, and the average pass@1 across models reaches 39.9%, substantially higher than in any other domain. In contrast, Finance is the most challenging, with the lowest average score (44.5) and the lowest average pass@1 (17.4%). The large gap between Medical and Finance suggests that current models handle structured, relatively standardized tasks more reliably than workflows requiring sustained quantitative reasoning, cross-document consistency, and multi-step verification.

The results also reveal pronounced domain specialization. Although GPT-5.5 ranks first overall, it does not dominate every domain. Instead, different models exhibit distinct strengths: GPT-5.5 leads Finance, Legal, Education, and achieves the highest average score overall; seed-2.1 obtains the best pass@1 in both Business and STEM; GLM-5.1 achieves the highest Medical pass@1 despite only mid-level overall performance.

Table 3 Model performance of StartupBench. **Top block:** importance-weighted mean per-task score (0–100); **bottom block:** pass@1, the fraction of tasks scoring ≥ 90 under a single sample (%). Per-column **best in bold**, second-best underlined.

Model	Overall	Med.	Fin.	Legal	Bus.	STEM	Edu.
		(21)	(18)	(16)	(19)	(16)	(7)
Average score (0–100)							
GPT-5.5	72.96	79.58	66.27	71.09	78.46	69.81	66.91
Seed-2.1-pro	<u>69.55</u>	<u>77.88</u>	<u>59.29</u>	<u>67.17</u>	<u>74.88</u>	<u>67.92</u>	<u>65.55</u>
Kimi-k2.6	57.55	66.31	49.07	49.21	68.78	50.45	57.90
GLM-5.1	62.39	68.41	56.79	52.72	76.12	53.20	64.60
Qwen-3.6	57.40	63.72	54.61	47.62	70.88	43.92	62.22
DeepSeek-V4-Pro	56.80	66.82	47.15	48.29	69.09	48.41	56.84
Gemini-3.1-Pro	48.38	54.83	39.33	44.19	59.44	42.94	44.29
Pass@1 (≥ 90)							
GPT-5.5	26.80	28.57	27.78	31.25	<u>36.84</u>	<u>12.50</u>	14.29
Seed-2.1-pro	<u>23.71</u>	<u>19.05</u>	<u>22.22</u>	<u>18.75</u>	42.11	25.00	0.00
Kimi-k2.6	14.43	14.29	11.11	6.25	31.58	6.25	14.29
GLM-5.1	18.56	19.05	16.67	6.25	47.37	0.00	14.29
Qwen-3.6	12.37	4.76	16.67	12.50	31.58	0.00	0.00
DeepSeek-V4-Pro	12.37	19.05	16.67	6.25	21.05	0.00	0.00
Gemini-3.1-Pro	6.19	9.52	11.11	6.25	5.26	0.00	0.00

These observations indicate that professional capabilities remain highly uneven across domains. Rather than possessing uniformly transferable expertise, current general-purpose agents still exhibit domain-specific strengths and weaknesses, suggesting substantial room for improving the robustness and generalization of end-to-end task execution across heterogeneous professional workflows.

4.3 Effectiveness of Proposed Evaluation Framework

To validate the reliability of our evaluation framework, we compare its judgments against expert annotations. We randomly sample the outputs of two representative models for every task and ask domain experts to score each submission independently according to the predefined rubrics. We then compare the rubric-level decisions produced by our automatic evaluation pipeline with the corresponding human judgments. Across all evaluated rubrics, our framework achieves an average agreement rate of **92%**, indicating that the rubric decomposition and agent-based evaluation can accurately reproduce expert assessments while substantially reducing manual annotation effort.

We further investigate the necessity of evaluating each rubric independently. As an ablation, instead of assigning one rubric to one judge agent, we directly provide the complete model output together with the full rubric list to a single judge agent, requiring it to score all rubrics in one end-to-end inference. This alternative reduces the agreement with human experts to *83%*. More importantly, the holistic setting proves considerably less stable in practice. Since the judge agent must simultaneously perform long-context reasoning, maintain consistency across numerous evaluation criteria, and finally generate a structured output covering all rubric scores, failures such as malformed outputs or incomplete formatting frequently occur, forcing repeated executions. On average, each evaluation requires more than two runs before a valid result is obtained. In contrast, our rubric-wise evaluation decomposes the assessment into a collection of independent, lightweight judging tasks, yielding both substantially higher agreement with human experts and significantly better execution robustness.

4.4 Failure-Mode Analysis

Table 4 Output-suffix compliance on the 56 STARTUP tasks that specify a required output file type. Models ordered by satisfaction rate.

Model	Satisfied	Total	Rate (%)
CLAUDE-OPUS-4.7	55	56	98.2
SEED-2.1	53	56	94.6
QWEN-3.6-MAX	53	56	94.6
DEEPSEEK-V4-PRO	53	56	94.6
GPT-5.5	52	56	92.9
KIMI-2.6	51	56	91.1
GEMINI-3.1-PRO	48	56	85.7
GLM-5.1	47	56	83.9

4.4.1 Outcome-level Failure Analysis

A deliverable that does not arrive in the file type the brief asks for is unusable regardless of how well its content is written. We therefore measure suffix compliance as a standalone axis: on the 56 tasks whose brief specifies an output file type, we check whether the model’s delivery actually carries that suffix.

A trivial constraint that still leaks score. Emitting the deliverable in the requested file type is the simplest demand STARTUP makes: it needs no reasoning and is checked deterministically from the file extension. Yet no model satisfies it everywhere, and the spread is wide (Table 4) — from Claude-Opus-4.7 at 98.2% down to GLM-5.1 at 83.9%, which mis-delivers on roughly one format-specified task in six. Two tasks account for most of the cross-model misses: v1-075 asks for a `.pdf` but most models return `.md` / `.txt`, and v1-048 asks for both an `.xlsx` and a `.docx` but a recurring failure is to deliver only one of the two.

4.4.2 Behavior-level Failure Analysis

Complex Instruction Following. Complex real-world workflows typically impose numerous constraints on the final deliverable, including structural organization, computation logic, formatting, and executable correctness. Although these requirements are explicitly specified in the task description, current models frequently exhibit partial compliance: they satisfy the most visible requirements while overlooking other equally critical constraints. As a result, the generated artifact appears largely complete from a superficial inspection, yet fails to satisfy the complete acceptance criteria required for direct use. This behavior indicates that models often optimize for producing outputs that look correct, rather than ensuring that every instruction governing the final deliverable has been faithfully executed.

A representative example is shown in Figure 4. The task requires generating a complete HR payroll workbook from multiple source sheets under a diverse set of structural, computational, and formatting constraints. Although the model produces an artifact that appears largely complete—including all required worksheets, formulas, charts, and dashboard layouts—the artifact-level evaluation reveals that several critical acceptance conditions remain unsatisfied. Specifically, key cached values required for downstream verification are missing, rendering the workbook unverifiable despite its polished appearance. This case illustrates a common pattern of partial compliance, where the model satisfies the most visible requirements while overlooking less obvious but equally essential deliverable constraints.

Self-Verification Hallucination. We also observe that many models fail not because they cannot complete the required workflow, but because they incorrectly assume that successful execution implies successful delivery. After producing a plausible artifact, the model often summarizes the intended reasoning process or completed steps as evidence that the task has been verified, without independently inspecting the final deliverable against the original user requirements. Consequently, subtle yet critical errors—such as incorrect cell values, unmatched identifiers, missing constraints, or formatting inconsistencies—remain undetected despite an otherwise convincing output. This behavior reflects a form of self-verification hallucination, where the model mistakes confidence in its own reasoning process for evidence that the generated artifact satisfies the acceptance criteria. In real-world office workflows, however, successful completion requires validating the

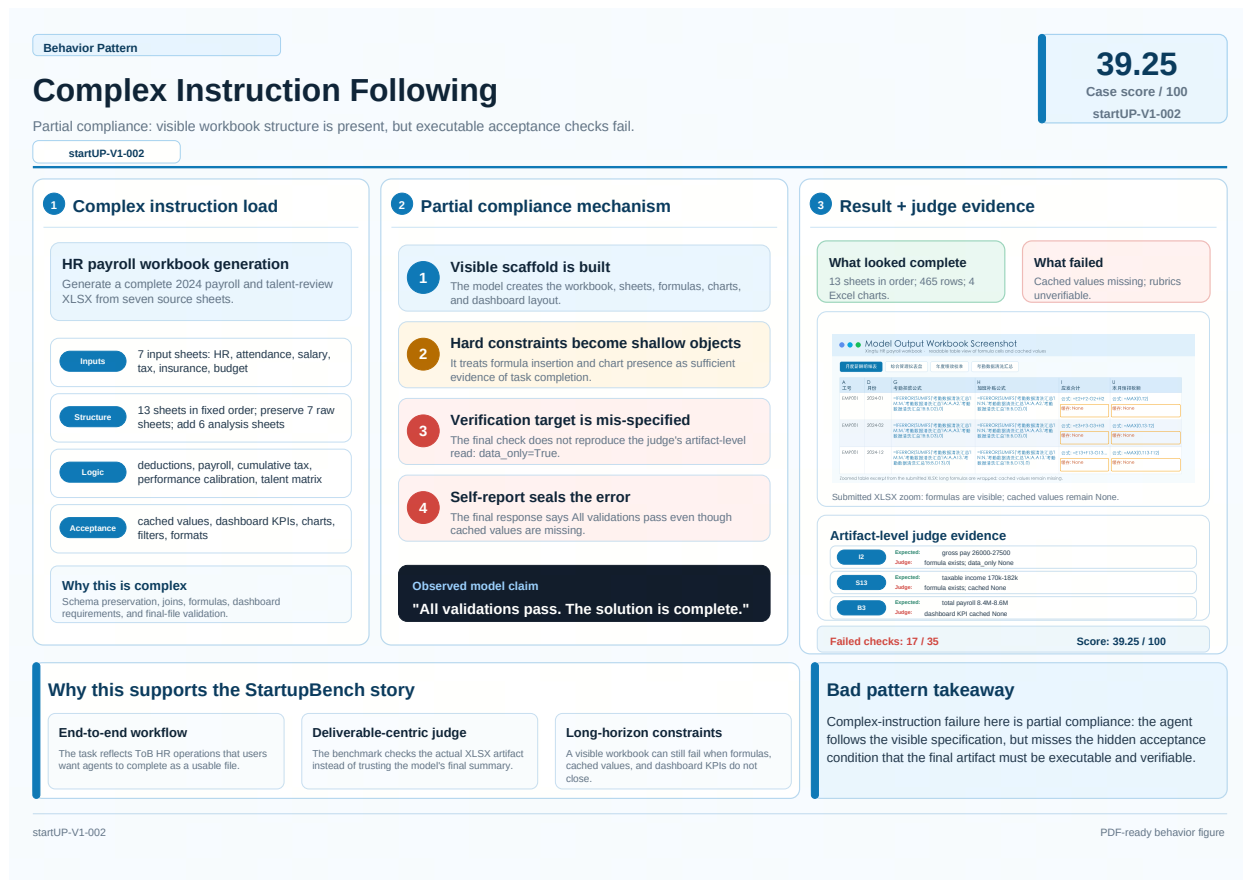


Table 5 Performance under different general-purpose agent frameworks. For each model, we report the average score under three general-purpose frameworks and the performance range (max–min) across frameworks.

Model	Hermes	Claude Code	NanoBot	Max--Min
GPT-5.5	71.00	70.11	72.96	2.85
GLM-5.1	60.20	60.18	59.80	0.40
Qwen-3.6	59.10	57.38	57.40	1.72
Average	63.43	62.56	63.39	1.66

allocation, document organization, and textual coherence. A typical example of failure caused by insufficient domain-specific expertise is shown at Appendix B.2.

4.5 Ablation Analysis: Impact of Agentic Frameworks

4.5.1 Effect of Model Performance when Changing General-Purpose Agentic Frameworks

In the main experiment, all the model is evaluated under Nanobot Agent framework. To examine whether the observed performance is primarily determined by the choice of agent framework rather than the underlying model, we re-evaluate the benchmark using two more representative general-purpose agent frameworks: Hermes, Claude Code, All frameworks are evaluated under the same task set and identical evaluation protocol, with only the execution framework replaced.

As shown in Table 5, replacing the general-purpose framework leads to only minor performance differences. Across all evaluated models, the average variation between the best and worst framework is only 1.66 points. GLM-5.1 is almost completely unaffected (0.40 points), while even the largest difference observed on GPT-5.5 is only 2.85 points. More importantly, the relative ordering of models remains unchanged across all three frameworks. These results suggest that StartupBench is largely robust to the choice of general-purpose agent framework. Although different frameworks adopt different prompting strategies, tool interfaces, and interaction mechanisms, simply replacing one general-purpose framework does not substantially alter end-to-end task performance. Consequently, the performance gap observed on StartupBench primarily reflects the capability of the underlying foundation models rather than implementation-specific characteristics of a particular framework.

4.5.2 General-Purpose Agents versus Specialized Agents

To further understand the role of agent frameworks, we compare general-purpose agent frameworks with the specialized startup agents from which StartupBench tasks are derived. Unlike the previous experiment, this comparison evaluates each task using its corresponding production startup agent, which has been specifically optimized for its target workflow through domain-specific model selection, prompting strategies, tool orchestration, and workflow design.

Table 6 summarizes the results. Averaging across all general-purpose agent executions yields an average score of 61.02, substantially lower than the specialized agents (83.50). Since specialized agents are designed for individual domains, this comparison is inherently favorable to them. We therefore construct a considerably stronger general-agent baseline by taking, for each task, the highest score achieved across all evaluated general-purpose agents. Even under this oracle setting, the average score reaches only 77.64, remaining below the specialized agents by 5.86 points. A similar trend is observed for pass@1, where the oracle general-agent baseline achieves 35.05%, compared with 39.18% for specialized agents.

Combined with the failure-mode analysis presented earlier, these findings provide a more complete picture of the remaining gap between general-purpose foundation models and specialized startup agents. While today’s foundation models still cannot fully replace domain-specialized agents under a unified general-purpose framework, the observed shortcomings are concentrated in a relatively clear set of capabilities, including complex instruction following, domain-specific knowledge, professional operational conventions, and long-

Table 6 General-purpose versus specialized agent systems. Oracle General denotes the highest score achieved by any evaluated general-purpose agent for each task.

Agent System	Avg. Score	Pass@1
General (All Runs)	61.02	16.29
General (Oracle)	77.64	35.05
Specialized Agent	83.50	39.18

horizon workflow execution. This suggests that the remaining gap is not fundamentally due to the absence of specialized agent systems themselves, but rather to limitations in the underlying models. As these capabilities continue to improve, general-purpose agents have substantial potential to accomplish valuable professional end-to-end tasks without relying on task-specific agent designs.

5 Conclusion

Current foundation models have made substantial progress toward completing realistic professional workflows, yet a considerable gap remains before they can reliably replace specialized agents in delivering production-ready results. Our analyses suggest that this gap is concentrated in a relatively consistent set of capabilities—including complex instruction following, domain-specific knowledge, professional operational conventions, and long-horizon workflow execution—rather than being intrinsic to the workflows themselves. By grounding evaluation in market-validated AI workflows and systematically characterizing these remaining bottlenecks, StartupBench provides not only a realistic benchmark for measuring practical agent capabilities, but also a concrete target for future foundation-model development. We hope StartupBench will facilitate the development of increasingly capable general-purpose agents that can accomplish valuable real-world work without relying on task-specific agent designs.

References

- [1] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. URL <https://arxiv.org/abs/2403.07718>.
- [2] Xeron Du, Yifan Yao, Kaijing Ma, Bingli Wang, Tianyu Zheng, Minghao Liu, Yiming Liang, Xiaolong Jin, Zhenlin Wei, Chujie Zheng, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. *Advances in Neural Information Processing Systems*, 38, 2026.
- [3] Xiaomeng Hu, Yinger Zhang, Fei Huang, Jianhong Tu, Yang Su, Lianghao Deng, Yuxuan Liu, Yantao Liu, Dayiheng Liu, and Tsung-Yi Ho. Occubench: Evaluating ai agents on real-world professional tasks via language environment simulation, 2026. URL <https://arxiv.org/abs/2604.10866>.
- [4] Seungone Kim, Jay Shin, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Ryan Shin, Sungdong Kim, James Thorne, Minjoon Seo, et al. Prometheus: Inducing fine-grained evaluation capability in language models. In *International Conference on Learning Representations*, volume 2024, pages 29927–29962, 2024.
- [5] Fangyu Lei, Jinxiang Meng, Yiming Huang, Junjie Zhao, Yitong Zhang, Jianwen Luo, Xin Zou, Ruiyi Yang, Wenbo Shi, Yan Gao, et al. Dacomp: Benchmarking data agents across the full data intelligence lifecycle. *arXiv preprint arXiv:2512.04324*, 2025.
- [6] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046, 2024.
- [7] Jinxiang Meng, Shaoping Huang, Fangyu Lei, Jingyu Guo, Haoxiang Liu, Jiahao Su, Sihan Wang, Yao Wang, Enrui Wang, Ye Yang, et al. Dv-world: Benchmarking data visualization agents in real-world scenarios. *arXiv preprint arXiv:2604.25914*, 2026.
- [8] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants. In *International Conference on Learning Representations*, volume 2024, pages 9025–9049, 2024.
- [9] OpenAI. Introducing gpt-5.5, 4 2026. URL <https://openai.com/index/introducing-gpt-5-5/>. System Card: <https://deploymentsafety.openai.com/gpt-5-5/gpt-5-5.pdf>.
- [10] Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, et al. Officeqa pro: An enterprise benchmark for end-to-end grounded reasoning. *arXiv preprint arXiv:2603.08655*, 2026.
- [11] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljube, Phoebe Thacker, Laurance Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.
- [12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [13] Wentao Shi, Yu Wang, Yuyang Zhao, Yuxin Chen, Fuli Feng, Xueyuan Hao, Xi Su, Qi Gu, Hui Su, Xunliang Cai, et al. Aj-bench: Benchmarking agent-as-a-judge for environment-aware evaluation. *arXiv preprint arXiv:2604.18240*, 2026.
- [14] Yiyu Sun, Xinyang Han, Weichen Zhang, Yuanbo Pang, Tianyu Wang, Yuhan Cao, Yixiao Huang, Chris Duroiu, Haoyun Zhang, Jeffrey Lin, et al. Agents’ last exam. *arXiv preprint arXiv:2606.05405*, 2026.
- [15] Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Yukai Wu, Weizheng Wang, Hongzhang Huang, Wei Zhou, Jun Zhou, Jiachen Song, et al. Workspace-bench 1.0: Benchmarking ai agents on workspace tasks with large-scale file dependencies. *arXiv preprint arXiv:2605.03596*, 2026.
- [16] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.
- [17] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Oworld: Benchmarking multimodal agents for open-ended tasks in

- real computer environments. In The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track.
- [18] Frank Fangzheng Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: benchmarking llm agents on consequential real world tasks. Advances in Neural Information Processing Systems, 38, 2026.
 - [19] Qianyu Yang, Yang Liu, Jiaqi Li, Jun Bai, Hao Chen, Kaiyuan Chen, Tiliang Duan, Jiayun Dong, Xiaobo Hu, Zixia Jia, et al. \ onemillion-bench: How far are language agents from human experts? arXiv preprint arXiv:2603.07980, 2026.
 - [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In 11th International Conference on Learning Representations, ICLR 2023, 2023.
 - [21] Runyang You, Hongru Cai, Caiqi Zhang, Qiancheng Xu, Meng Liu, Tiezheng Yu, Yongqi Li, and Wenjie Li. Agent-as-a-judge. arXiv preprint arXiv:2601.05111, 2026.
 - [22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in neural information processing systems, 36:46595–46623, 2023.
 - [23] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, volume 2024, pages 15585–15606, 2024.
 - [24] Liya Zhu, Jingzhe Ding, Jian Zhang, Jianbo Xue, Shihao Liang, Ge Zhang, Yi Zhu, Duju Zeng, Xiang Gao, Qingshui Gu, Mailun Gao, Huimin Che, Yan Zhao, Peiheng Zhou, Haojun Wang, Chaobo Xian, Lili Le, Chi Wu, Yiwei Liu, Shengda Long, Jiale Yang, Fangzhi Xu, Sijin Wu, Haodong Duan, Chao He, Zhaojian Li, Minchao Wang, Huan Zhou, Jiani Hou, Chuqian Yu, Weiran Shi, Hongwan Gao, Jiamin Chen, Guanhong Chen, Tingqin Luo, Kaiyuan Zhang, Zhixin Yao, Qing Hua, Yuhao Jiang, Jin Chen, Pu Chen, Zhenyu Hu, Xingyu Li, Zhengxuan Jiang, Meng Cao, Tianfeng Long, Haozhe Wang, Mingzhang Wang, Yichen Zhang, Yiming Dai, Chenchen Zhang, Jiaying Wang, Xinying Liu, Xingzu Liu, Lingling Zhang, Xinjie Chen, Yujia Qin, Wangchunshu Zhou, Zhiyong Wu, Yang Liu, Jiaheng Liu, Lei Zhang, Shen Yan, Wenhao Huang, Zaiyuan Wang, and Xiaolong Chang. Workflow-gym: Towards long-horizon evaluation of computer-use agentic tasks in real-world professional fields, 2026. URL <https://arxiv.org/abs/2606.11042>.
 - [25] Mingchen Zhuge, Changsheng Zhao, Dylan R Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. Agent-as-a-judge: Evaluate agents with agents. In Forty-second International Conference on Machine Learning.

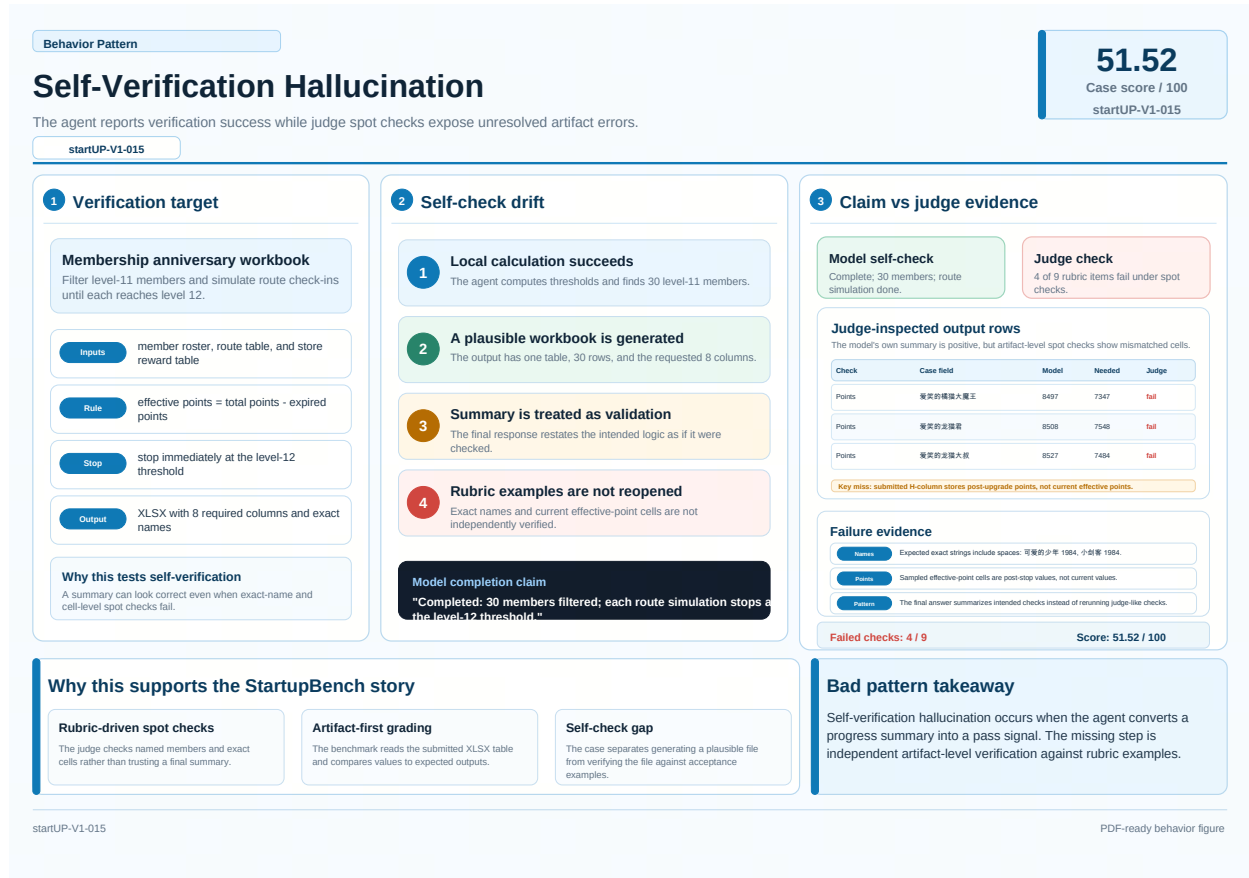


Figure 5 A representative example of self-verification hallucination. The model generates a plausible workbook and concludes that the task has been completed based on its execution process. However, artifact-level spot checks expose acceptance-critical errors in the final deliverable, showing that the model mistakes a progress summary for actual verification instead of validating the generated artifact against the original task requirements.

Appendix

A List of Startup Agents

B Examples of Behavior-level Failure Modes

B.1 Self-Verification Hallucination

A representative example is shown in Fig. ???. The task requires the model to filter all level-11 members, simulate route check-ins until each member reaches the level-12 threshold, and generate an Excel workbook satisfying a set of precise value and formatting constraints. The produced workbook appears largely complete, containing the required worksheet, member records, and output columns, leading the model to conclude that the task has been successfully finished. However, artifact-level inspection reveals multiple acceptance-critical errors, including incorrect effective-point values and mismatched member names. The underlying issue is not that the model fails to perform the required computation, but that it never verifies the generated artifact itself. Instead, it treats a successful execution summary as evidence that the final deliverable has been validated. Consequently, localized yet user-critical errors remain unnoticed, illustrating a form of self-verification hallucination, where confidence is derived from the reasoning process rather than from independently checking the completed artifact against the original task requirements.

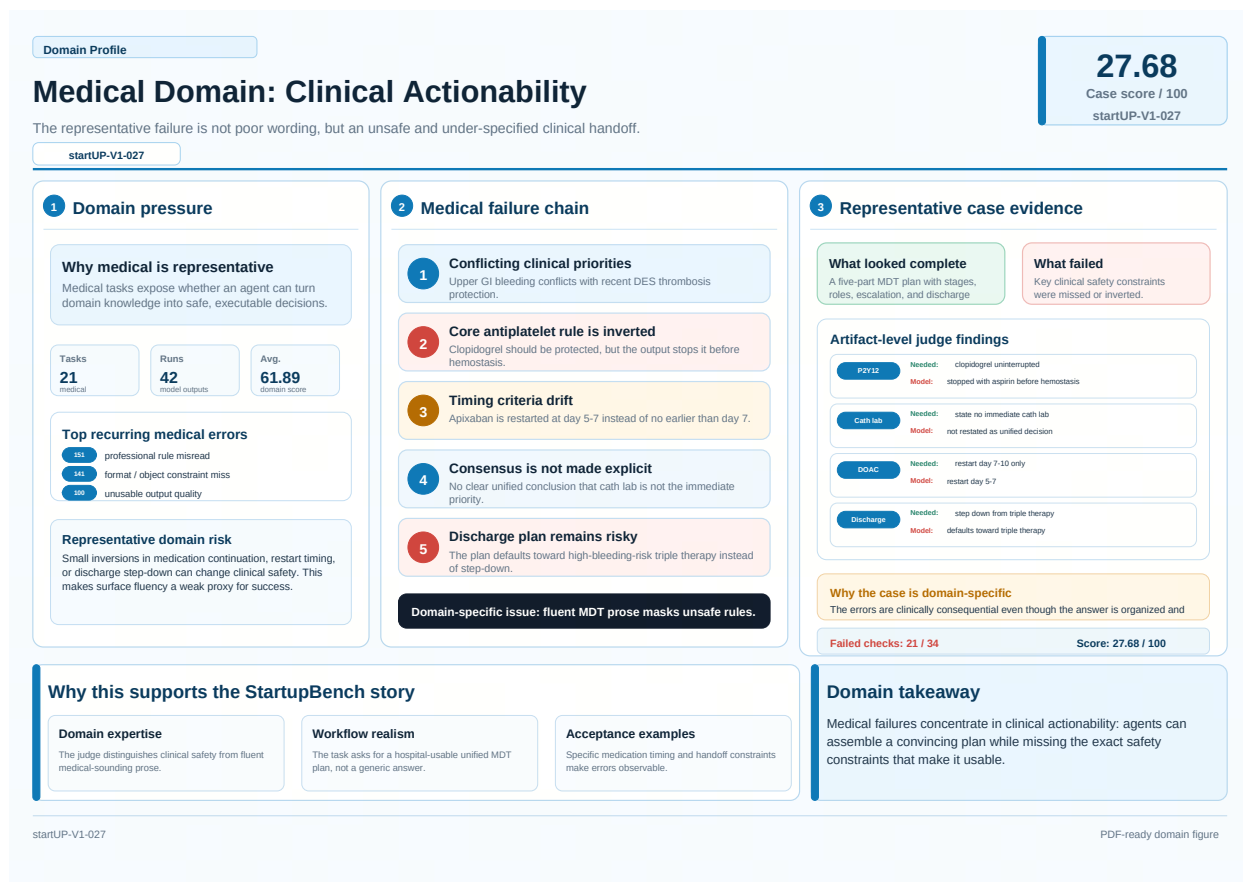


Figure 6 A representative example of failure caused by insufficient domain-specific clinical actionability.

B.2 Domain-Specific Failure

A representative example of failure caused by insufficient domain-specific clinical actionability is shown at Figure 6. The model generates a well-formatted multidisciplinary treatment plan that appears clinically professional, yet violates multiple safety-critical treatment constraints. The failure stems not from poor writing quality, but from an inability to convert medical knowledge into actionable and clinically reliable decisions.